PYFORGE

Cursor-ready project specification & implementation blueprint

Project: Production-minded Python Engineering Toolkit
Specification version: 1.0
Date: 2 November 2026
Owner: Suhas G S
Target: A maintainable, typed, tested Python package suitable for reuse in future AI/ML and backend systems.

Purpose of this document
Give this PDF to Cursor as the source of truth for implementing PyForge. Cursor should inspect the existing starter repository, compare it against this specification, then implement the missing work in small, testable increments. It must not replace the project with an unrelated template or silently claim features are complete.

Engineering principle: reliability first, abstraction second. PyForge should remain small, explicit, and useful. It is not intended to become a framework that wraps every Python problem.

0. Instructions for Cursor

Treat this document as the project requirements and acceptance contract. Before editing files, inspect the repository and report what already exists, what is incomplete, and any conflicts with this specification.

1. **Audit first.** Inspect the complete repository, Python version, pyproject.toml, source files, tests, CI, documentation, and Git status.
2. **Preserve working code.** Do not delete or rewrite functioning modules merely to impose a preferred style. Make focused changes.
3. **Plan before implementation.** Produce a short checklist grouped by milestone, identify dependencies, then implement Milestone 0 and Milestone 1.
4. **Work incrementally.** After each milestone, run formatting, lint, type checks, and tests. Fix failures before proceeding.
5. **Do not fabricate.** Never invent benchmark numbers, coverage claims, security guarantees, release status, or completed features.
6. **Document decisions.** Add or update architecture documentation and ADRs when public APIs, dependencies, or design trade-offs change.
7. **Ask only when blocked.** If a decision is genuinely ambiguous and affects architecture or compatibility, explain the options and ask. Otherwise choose the simplest safe default and document it.
8. **Finish with evidence.** Report changed files, commands executed, pass/fail results, known limitations, and the exact next milestone.

Cursor operating rule: Do not attempt to implement the entire long-term roadmap in one unreviewed pass. Build a working, tested increment first.

1. Product overview

1.1 Product vision

PyForge is a small, reusable Python engineering toolkit that demonstrates practical software-engineering discipline. Its first release focuses on resilience primitives that future applications can reuse: bounded retries, structured logging, configuration handling, and safe HTTP request patterns.

1.2 Primary objectives

- Provide a clean, typed, documented Python package with a stable public API.
- Implement predictable synchronous and asynchronous retry behavior.
- Make time, randomness, and external behavior injectable so tests are deterministic.
- Provide structured logging and configuration helpers without unnecessary runtime dependencies.
- Demonstrate packaging, CI, linting, type checking, coverage, and release discipline.
- Serve as a reusable foundation for later AI/GenAI projects, not as an AI product itself.

1.3 Non-goals

- Not a web framework, task queue, workflow engine, distributed scheduler, or observability platform.
- Not a replacement for mature libraries such as Tenacity, HTTPX, structlog, or Pydantic.
- No automatic retries of arbitrary operations without explicit caller configuration.
- No credentials, telemetry, network calls, or background activity by default.
- No claims of production certification or security guarantees.

2. Target users and core use cases

User	Need	PyForge value
Python application developer	Make transient failures recoverable	Explicit bounded retry policies
AI/ML engineer	Make API and model-service calls more robust	Reusable resilience helpers and structured logs
Backend engineer	Standardise configuration and diagnostics	Typed helpers and predictable behavior
Maintainer	Keep behavior testable and stable	Deterministic tests, CI, docs, semantic versioning

3. Technology and compatibility

Area	Requirement
Language	Python 3.11+
Packaging	PEP 517/518 with setuptools; src layout
Runtime dependencies	Prefer Python standard library for core modules
Development dependencies	pytest, pytest-asyncio, pytest-cov, Ruff, mypy
Testing	pytest; deterministic unit tests; integration tests where justified
Quality	Ruff lint + format, mypy strict mode, coverage report
CI	GitHub Actions on Python 3.11, 3.12, and 3.13
License	MIT unless owner changes it
Documentation	Markdown in README.md and docs/

Pin minimum supported versions in pyproject.toml. Avoid adding runtime dependencies unless a documented feature requires them. Every dependency must have a reason.

4. Functional requirements

4.1 Retry policy

Implement an immutable RetryPolicy configuration object with these fields:

Field	Type	Default	Rules
max_attempts	int	3	Includes initial call; minimum 1
initial_delay	float	0.25	Seconds; >= 0
max_delay	float	5.0	Seconds; >= 0
multiplier	float	2.0	>= 1
jitter	float	0.0	Fraction in [0, 1]

- Calculate exponential backoff after each failed attempt and cap it at max_delay.
- Apply symmetric bounded jitter using an injected random value in [0, 1].
- Reject invalid policy values immediately with clear ValueError messages.
- Expose a documented delay calculation method for tests and callers where appropriate.

- Ensure a zero delay is valid and does not cause unexpected behavior.

4.2 Retry decorator

Provide a decorator factory supporting both synchronous and asynchronous callables.

- Accept an explicit tuple of retryable Exception subclasses. Validate the tuple.
- Retry only configured exception classes. Non-matching exceptions propagate immediately.
- The final matching exception must be re-raised unchanged, preserving its type and message.
- Preserve wrapped function metadata using functools.wraps.
- Allow sleep and random-value providers to be injected for deterministic tests.
- For async functions, use awaitable sleep and preserve cancellation behavior. asyncio.CancelledError must never be swallowed or retried.
- Do not catch BaseException.
- Document that retrying a remote write is safe only when the operation is idempotent or protected by an idempotency key.

4.3 Structured logging

Add a small structured logging module in a later milestone. It should use the standard logging package and provide consistent fields without forcing global logging configuration.

- Provide a JSON formatter or JSON serialization helper for log records.
- Include timestamp, level, logger name, message, and optional exception details.
- Allow callers to attach contextual fields such as request_id, operation, attempt, and component.
- Never log secrets, authorization headers, passwords, tokens, or raw sensitive payloads.
- Do not configure the root logger automatically when the package is imported.

4.4 Configuration helpers

- Provide explicit helpers for reading environment-backed settings.
- Support defaults, required values, and basic type conversion.
- Raise clear errors for missing or invalid required configuration.
- Do not read .env files unless an optional dependency and explicit opt-in are introduced.
- Never print or log secret values.

4.5 HTTP resilience example

- Provide an optional example or integration module showing retries around transient HTTP failures.
- Keep HTTP client dependencies optional; do not add them to the core runtime dependency set without approval.
- Retry only safe methods or requests explicitly configured as retryable.
- Respect caller-provided timeouts and cancellation.
- Demonstrate idempotency keys for retryable write operations.

5. Public API expectations

The public API should be small and stable. Start with the current RetryPolicy and retry symbols, then expose additional helpers only after their behavior and tests are settled.

```
from pyforge import RetryPolicy, retry
```

```
@retry(
    policy=RetryPolicy(
        max_attempts=4,
        initial_delay=0.25,
        max_delay=2.0,
        multiplier=2.0,
        jitter=0.1,
    ),
    retry_on=(TimeoutError,),
)
def fetch_data() -> dict[str, str]:
    return {"status": "ok"}
```

API design requirements:

- All public functions, classes, and methods must have type annotations.
- Public objects must have useful docstrings and examples.
- Avoid exposing internal implementation details unnecessarily.
- Do not break an existing public API without a documented migration note.
- Use explicit names and avoid overly generic abstractions.

6. Architecture and repository layout

```
pyforge/
■■■ src/
■   ■■■ pyforge/
■       ■■■ __init__.py
■       ■■■ py.typed
■       ■■■ retry.py
■       ■■■ logging.py          # Milestone 2
■       ■■■ config.py           # Milestone 3
■       ■■■ http/               # Milestone 4, only if justified
■■■ tests/
■   ■■■ unit/
■   ■■■ integration/
■   ■■■ conftest.py
■■■ examples/
■■■ docs/
■   ■■■ architecture.md
■   ■■■ api-reference.md
■   ■■■ security.md
■   ■■■ decisions/
■■■ .github/
■   ■■■ workflows/
■       ■■■ ci.yml
■■■ pyproject.toml
■■■ README.md
■■■ CONTRIBUTING.md
■■■ CHANGELOG.md
■■■ LICENSE
■■■ .gitignore
```

Keep the initial repository smaller if the modules do not exist yet. Create directories when they contain real files; do not create empty scaffolding solely to make the tree look impressive.

7. Reliability, safety, and security requirements

- **Idempotency:** prominently document that retrying a non-idempotent operation can duplicate side effects.
- **Exception boundaries:** catch only configured Exception subclasses; never catch BaseException.
- **Cancellation:** async cancellation must propagate immediately.
- **Bounds:** attempts and delays must always be bounded by validated policy values.
- **Secrets:** never commit .env files, tokens, credentials, or customer data.
- **Logging:** redact or omit secret-bearing fields and avoid logging raw request bodies by default.

- **Imports:** importing pyforge must not change root logging configuration or perform network activity.
- **Dependencies:** review dependency necessity and avoid supply-chain bloat.

8. Testing strategy

8.1 Required retry tests

- Policy accepts valid boundary values and rejects invalid values.
- Exponential delay calculation is correct and capped.
- Jitter lower and upper bounds are correct.
- A synchronous operation succeeds on its first attempt.
- A synchronous operation fails transiently and then succeeds.
- The final configured exception is re-raised after exactly max_attempts calls.
- A non-retryable exception propagates after one call.
- A function's name, docstring, and metadata are preserved.
- An asynchronous operation retries and succeeds.
- Async cancellation propagates and is not retried.
- Injected sleep is called with expected delays and no real waiting occurs.
- Invalid retry_on values are rejected.

8.2 Quality gates

Gate	Acceptance
Ruff lint	No lint errors
Ruff format	All tracked Python files formatted
mypy	Strict type check passes for src
pytest	All unit and integration tests pass
Coverage	At least 90% statement coverage for core package; branch gaps reviewed
CI matrix	All supported Python versions pass
Build	Package builds successfully using standard Python build tooling

Coverage is a signal, not the goal. Do not write meaningless tests solely to raise a percentage. Document any justified exclusions.

9. Documentation requirements

- README: purpose, status, supported Python versions, installation, quick start, examples, feature list, development commands, roadmap, limitations, and license.
- Architecture document: module boundaries, control flow, error behavior, and known limitations.
- API reference: public classes, parameters, return values, raised exceptions, and examples.
- Security notes: idempotency, secret handling, logging risks, and dependency policy.
- ADR records: important decisions, context, alternatives, and consequences.
- Changelog: user-visible changes by release.

- Contributing guide: setup, checks, tests, pull request expectations.

10. CI/CD and release process

10.1 Continuous integration

- Run on pushes and pull requests targeting main.
- Use a Python version matrix: 3.11, 3.12, 3.13.
- Install the package and development dependencies from pyproject.toml.
- Run Ruff lint, Ruff format check, mypy strict, and pytest with coverage.
- Use least-privilege workflow permissions.
- Pin third-party GitHub Actions to reviewed major versions at minimum; use commit SHA pinning where the maintainer's policy requires it.

10.2 Release readiness

- Version follows semantic versioning once a stable public API exists.
- Changelog reflects actual user-visible changes.
- Build artifacts are generated and inspected.
- Tests pass on the supported Python matrix.
- README installation instructions are verified in a clean environment.
- Do not publish to PyPI or create a GitHub release without explicit owner approval.

11. Milestones and deliverables

Milestone	Scope	Exit criteria
M0: Audit	Inspect starter repo; identify gaps and conflicts	Written audit and implementation checklist
M1: Retry core	Policy, sync/async decorator, validation, tests	All retry acceptance tests pass
M2: Logging	Structured formatter/helper, redaction guidance	Tests cover formatting and sensitive-field behavior
M3: Configuration	Environment-backed typed settings helpers	Defaults, required values, conversion, errors tested
M4: HTTP example	Optional HTTP resilience integration	Safe retry semantics and idempotency explained
M5: Quality	Coverage, type checks, build, docs, CI hardening	All gates pass on supported matrix
M6: Release	Changelog, version, packaging verification	Owner reviews and approves release

12. Definition of done

- A fresh clone can be installed using documented commands.
- Core package behavior is covered by deterministic tests.
- All public APIs are typed and documented.
- Lint, format, type-check, tests, coverage, and build checks pass.
- CI runs automatically for pull requests and pushes to main.
- No secrets or local environment files are committed.

- Architecture, limitations, and key design decisions are documented.
- README claims match implemented behavior.
- Known limitations and future work are listed honestly.

13. Cursor task prompt

Paste the following prompt into Cursor after opening the PyForge repository. Keep this PDF available in the workspace as the requirements document.

```
You are the senior Python engineer responsible for implementing PyForge.

Read the complete PyForge Cursor Implementation Specification PDF before
changing code. Treat it as the source of truth.

FIRST:
1. Inspect the entire repository and Git status.
2. Compare the existing implementation with every relevant requirement.
3. Report the current state, gaps, risks, and a milestone-based plan.
4. Do not overwrite working code or introduce unnecessary dependencies.

THEN:
5. Implement Milestone 0 (audit) and Milestone 1 (retry core) only.
6. Add or improve deterministic tests for sync and async behavior,
   retryable versus non-retryable exceptions, max attempts, backoff,
   jitter, invalid configuration, metadata preservation, and cancellation.
7. Ensure asyncio cancellation is propagated and BaseException is not caught.
8. Update pyproject.toml so every test dependency used by CI is declared.
9. Run Ruff lint, Ruff format check, mypy strict, pytest with coverage,
   and a package build. Fix failures rather than hiding or skipping them.
10. Update README and architecture documentation to match actual behavior.

CONSTRAINTS:
- Do not claim tests pass unless you ran them and observed the result.
- Do not fabricate coverage, benchmarks, or production-readiness claims.
- Do not add a runtime dependency without explaining why.
- Do not publish a package or create a release.
- Do not start Milestone 2 until Milestone 1 passes its acceptance criteria
  and you provide a concise implementation report.

At the end, report:
- files changed,
- features implemented,
- commands executed and exact results,
- remaining issues,
- known limitations,
- next recommended task.
```

14. Suggested local workflow

```
# 1. Create and activate a virtual environment
python -m venv .venv
# Windows PowerShell:
.venv\\Scripts\\Activate.ps1

# 2. Install the package and all development/test dependencies
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"

# 3. Run quality checks
ruff check .
ruff format --check .
mypy src
pytest --cov=pyforge --cov-report=term-missing

# 4. Build the package (install build first if needed)
python -m pip install build
python -m build
```

The development dependency group must include every tool used by local development and CI, including pytest-asyncio if async tests use it. Do not rely on undocumented manual installation steps.

15. Project owner decisions and assumptions

Decision	Current assumption
Project name	PyForge
Repository name	pyforge
GitHub owner	suhas781
Visibility	Public portfolio repository
Python support	3.11 and newer; CI tested on 3.11–3.13
Runtime dependencies	Standard library preferred
License	MIT
Initial scope	Retry core, followed by logging and configuration
Release authority	Owner approval required

Final instruction: Build a small, reliable, well-tested toolkit. Prefer explicit behavior over clever abstractions. The quality of the engineering decisions matters more than the number of modules.